Lab 8: Iterators and Functors

Due **Sunday, November 12, 2023 at 8:00PM ET** on Gradescope.

Each lab is graded for attendance (individual) and completion (group).
Note: Don't forget to add your pod members to the Gradescope submission!

Getting started

1. One of your group members should create an editable copy of this worksheet to fill out and share the document with all other group members. (*File → Make a Copy* in the menu.)
2. Fill in the table below based on the letters given to each group member. If your group does not have exactly four members, please work out a strategy for sharing the roles that works for you.

	Name	**Role (rotates each week)**
A	Spence	Coder I
B	Shikha Nukala	Coder II
C	Sonya Konon	Leader
D	All	Scribe

3. The ***scribe*** should do a find/replace (select *Edit > Find and replace* from the menu) to change all instances of "Spence" to that member's name, "Shikha" to that member's name, and so on.

Formatting
Please do not change the formatting of the document or which exercises appear on which slides. We use the placement of questions on each page to assist with grading.

Questions?
You are expected to lead the meeting yourselves and collaboratively work through the exercises, but please let us know if you have questions! We're here if you need us and will be checking in throughout the lab.

Note
These lab exercises were developed for in-person and virtual labs. Depending on the way you and your group are working, you are welcome to interpret the exercise instructions however works best (e.g. "share screen" means something different if you're in-person vs. virtual).

Lab Pod Breakout Questions

During lab, write down responses to each of the questions when you are prompted to do so by your lab instructor.

Question	**Discussion notes**
Why is learning implicit bias important for the work we do in tech?	A lot of algorithms in machine learning, data science, and AI are coded to evaluate user input and draw conclusions. If there is implicit bias in the conclusions drawn as written by the programmer, the algorithms could become internally biased which would outcast a population of users.

Turning Negatives into Positives

Each pod member will write down one typical negative thought they have in the table below under “negatives” (ex: I’m not good at school, I don’t try hard enough, Coding isn’t for me., etc). Then as a group discuss how you can change these negatives to positives and write down these alternative statements. See example below.

	Negatives :/	**Positives :)**
Example	**Coding isn’t for me.**	**Coding is a skill that can and must be learned like any other skill, and everyone has to learn it.**
Spence	**I suck at coding**	**I did well on the midterm.**
Shikha	**I cannot grasp how to code**	**If I really find an interest in what I am working on, I will grasp the material better.**
Sonya	**I feel like I failed the midterm.**	**I did not fail the midterm.**
All	**I cannot do the project.**	**I can go to office hours and start early to slowly grasp the objective and debug the code.**

Warm-up Questions

Consider a standard vector of ints, `std::vector<int>`. Like other containers, this data structure supports the iterator interface! Answer the following warm-up questions about iterators:

Question	**Discussion Leader**	**Your answer**

What is the type of an int vector iterator? That is, what would we have to type in the blank to get the following code to compile (besides `auto`!)
 	
`vector<int> my_vec = {1, 2, 3};`
`________ my_it = my_vec.begin();`
	Spence	The type would be It to initialize the beginning of the vector position as a iterator.

Iterators are defined as a nested class inside of the container that they are an iterator for. Why are the Iterator classes marked as public within their Container class?	Shikha	Must be public because the overloaded operator functions need to be accessible outside the class when working with actual vectors.
What does a `friend` declaration inside an Iterator class actually do?	Sonya	It gives outside class calls (i.e. declaring a class object) the ability to access private members of the Iterator class.
Can you think of a case when we wouldn't need a `friend` declaration inside of an Iterator class?	All	If the class to the outside class was inside the private block of the Iterator class, we wouldn't need to grant access to the private members of the class.

SpaceVector

In mission-critical operations such as space flight, it is common for computers to store redundant information or perform redundant computations to reduce the chance of error. Consider the following class, SpaceVector, which stores 3 copies of each element contiguously in case cosmic gamma rays cause parts of the computer memory to become corrupted. You can assume that all SpaceVector functions listed below are implemented, except for the Iterator functions, which you will implement.

```
class SpaceVector {
public:
  SpaceVector();

  void push_back(int elt);
  int size();

  class Iterator {
    public:
     Iterator();
     // Implement these functions below
      int& operator*() const;
      Iterator& operator++();
      bool operator==(Iterator rhs) const;

    private:
      int* ptr;
      // You may assume this variable points to the last
      // element in the elements array.
      int* const last_element;
      friend class SpaceVector;
  };

  Iterator begin() const;
  Iterator end() const;

private:
  const int CAPACITY = 20;
  int elements[CAPACITY];
  int size;
};
```

For example, after executing the following lines:

```
SpaceVector v = SpaceVector();
v.push_back(5);
v.push_back(2);
```

The number of elements in the `elements` array will be 6 and it will contain {5, 5, 5, 2, 2, 2}, but `v.size()` will return 2.
Implement the Iterator default constructor, dereference, increment, and equality operator according to their RME's.

HINT: Make sure to think about how you should represent the end iterator!

Discussion Leader	**Your answer**

Shikha	`// REQUIRES: this is a valid, dereferenceable iterator.`
	`// EFFECTS: returns the element this iterator points to`
	`int& operator*() const {`
	`  Return *((*this).ptr);`
	`}`

Sonya	`// REQUIRES: this is a valid, dereferenceable iterator.`
	`// EFFECTS: increments this iterator to point to the next`
	`// element in the SpaceVector`
	`Iterator& operator++() const {`
	` this->ptr +=3;`
	`}`

All	`// REQUIRES: this is a valid iterator.`
	`// EFFECTS: returns true if this is the same as rhs`
	`bool operator==(Iterator rhs) const {`
	` return this->ptr == rhs.ptr;`
	`}`

Functors, Predicates, and Higher-Order Functions

Question	Discussion Leader	Your answer
What operations can be performed on a first class object? Are functions first-class? How about classes?	Sonya	C++ does not have first-class functions. Classes and objects can be first-class. Can perform comparative operations on first class objects.
What is the difference between a functor and a function pointer? Why might we want to use a functor instead of a function pointer?	All	A function pointer is a variable that stores an address to a function. A functor is a class-type object that pretends to be the function by overloading the function call operator. It directly acts like a function while the pointer has to be called upon to act like a function.
What is a predicate? What must be true about the signature of the `()` operator for a functor for it to also be a predicate?	Spence	A predicate is a function that returns either true or false depending on whether its argument has some property. The signature must be const to be a predicate.

The EECS Department wants to create a list of all the students who have taken each EECS course in order to determine how many sections of each course to offer in the future.

Consider the following struct definition of a student:

```
struct Student {
  string uniqname;
  int UMID;
  vector<string> coursesTaken; //For example {"eecs280", "eecs203"}
};
```

We will write a functor class called `CheckClass` that can be used to create individual functor objects that each check whether a student has taken a particular class, specified when the functor is instantiated. Here's an example:

```
Student nishu = {
  "nishuk",
  12345678,
  {"engr101", "eecs203", "eecs201", "eecs280", "eecs370", "eecs281"}
};

CheckClass checker376("eecs376"); // create the functor

if(checker376(nishu)) { // use the functor
  std::cout << "Nishu has taken 376!" << endl;
}
else {
  // this one gets printed
  std::cout << "Nishu has not taken 376!" << endl;
}
```

Complete the missing portions of the `CheckClass` functor so that it works as in the example. Write your implementations in the table below.

```
class CheckClass() {
public:
  // Constructor
  // Overloaded () Operator
private:
  string course;
};
```

Question	**Discussion Leader**	**Your answer**
Write the code for the `CheckClass` constructor.	Shikha	CheckClass(string class) : course(class) {}

Write the code for the `CheckClass` overloaded `()` operator.	Sonya	Bool operator()(Student stu) {
		for(vector<string>::iterator i = stu.coursesTaken.begin(); i != stu.coursesTaken.end(); i++){
		If(course == *i) {
		Return true;
		}
		}
		Return false;

Now, let's write a function that uses your `CheckClass` functor to count the number of students who have taken a particular class:

```
// EFFECTS: Returns the number of students who have taken a course
int numTaken(const vector<Student> &students, const string &course);
```

However, functors are almost never used on their own. In general, they are most useful when paired either with a higher-order function (e.g. `any_of` from lecture, paired with a predicate) or with a class (e.g. `BinarySearchTree` from P5, paired with a comparator). So your `CheckClass` functor needs a partner. Assume we have this higher-order function:

```
// EFFECTS: Returns the number of elements specified by the
//          sequence [begin, end) that satisfy the predicate pred.
template <typename Iter, typename Pred>
int count_if(Iter begin, Iter end, Pred pred);
```

Now, use `CheckClass` and `count_if` together to implement `numTaken`. Your implementation code should not directly use a loop.

Question	**Discussion Leader**	**Your answer**

Write the code for the `numTaken` function.	All	Int numTaken(const vector<Student> &students, const string &course) {
		checkClass checker(course);
		Return count_if(students.begin(), students.end(), checker);

Bringing the Funk Back with Functors (and Triangles)

You may remember the `Triangle` struct from Lecture 7. In this example, we will focus on writing functors that will properly compare Triangles. For sake of simplicity, you do not need to be checking any triangle invariants in this exercise.

The **Scribe** will download the lab8_Triangle folder within the Lab 8 folder.

Before you start completing the code in the files, please answer the questions below.

Question	**Discussion Leader**	**Your answer**
What is the difference between a comparator and a predicate?	Spence	A predicate is a function that returns true or false based on if the argument has a certain property. Comparators evaluate two values against each other and return a true or false.
We have subtly worked with something similar to comparators in the past. Recall `Card_less` from p3? Why did we need to define multiple functions called `Card_less`? Why couldn't we just overload the comparison operators (e.g. `<`, `==`, etc.)	Shikha	We had to define multiple card less operators because there are different scenarios where we need to compare cards depending on if we have a trump suit yet or not.
Could the `Card_less` function (the one that considers both trump suit and led card) be used with a higher-order function like `any_of` from lecture? If so, how? If not, why not?	Sonya	There would be value to implementing the Card_less function as a higher-order function because every instance of the iteration through the player's hand we did could have been faster to implement.
What advantage would a comparator functor have over the `Card_less` function?	All	It would reduce the amount of code needed to implement the function and would be easier to check the condition in the euchre file.

Take a look in `lab09.hpp` to familiarize yourselves with the `Triangle` class. Though the `Triangle` class has a member variable called `area`, rest assured, you will not be doing any math in this lab. Your team will write the basic comparator `CompareArea` class, the predicate `IsAreaN`, and complete some other helper functions. Stubs for these functions are found in `lab08.cpp`. You will replace these with your code, which you will also write in the table below. The **Scribe** will share their screen and type the implementation for these functions. The **Discussion Leader** will start the discussion by sharing their ideas for the code. Then the entire group will collaborate to come to an agreement on the code.

Task	**Discussion Leader**	**Your Answer**
Complete the comparator for `CompareArea` in `lab09.cpp`. This comparator takes in two Triangles and returns true of the area of `a` is *less* than the area of `b`. Make sure to respect the interface for Triangle!	Spence	`return a.get_area() < b.get_area();`
Take a look at the `find_max()` function that is implemented for you. Why do we use the variable `comp` instead of just the regular `<` operator?	Shikha	Because the regular comparison operators are not overwritten for triangles
Implement the function `find_largest_triangle()` where the largest triangle is defined based on area. You may find the function `find_max()` useful.	Sonya	`CompareArea comp;` `return find_max(tri, num_triangles, comp);`

Task	Discussion Leader	Your Answer
Complete the definition for the predicate `IsAreaN`, which can be used to identify triangles with a specific area. Member variables are already commented out for you. Be sure to include any constructors or overloaded operators you think are necessary.	All	`bool operator()(const Triangle &tri, int area_in) const { return tri.get_area() == area_in; }`
Take a look at the function `print_if()`. Finish up the implementation for this function according to its RME.	Spence	`void print_if(const Triangle *arr, int n, Predicate pred) { for(int i = 0; i < n i++){ if(pred(arr[i])){ cout << arr[i].get_type << ", " << arr[i].get_area << endl; } } }`
Lastly, take a look at the function `print_triangle_with_area_n()`. Implement this function. You might find the function `print_if()` to be useful as well as the predicate `IsAreaN`.	Shikha	`void print_triangle_with_area_n(const Triangle *tri, int num_triangles, double n) { IsAreaN pred(n); print_if(tri, num_triangles, pred); // TASK 5 - YOUR CODE HERE }`

After completing each of these functions in `lab08.cpp`, run the provided testing code to verify your implementation. To compile and run the tests, run `make test` from your terminal. This compiles your code, runs main, and puts the output in a file called `lab08.out`. To check output correctness, run `diff lab08.out lab08.out.correct` .

CSE Diversity Report

Consider the table below from the 2022-2023 CSE Diversity report and answer the question below. You can reference the entire 2022-2023 CSE Diversity report here in your free time.

	Start of EECS 183, ENGR 101, ENGR 151				End of EECS 376			
	AY 2023	AY 2022	AY 2021	AY 2020	AY 2023	AY 2022	AY 2021	AY 2020
Man	53.76%	57.49%	59.76%	61.62%	68.02%	71.19%	67.89%	75.50%
Woman	44.08%	40.6%	39.45%	37.58%	29.07%	27.15%	31.41%	24.06%
Nonbinary	1.73%	1.6%	0.53%	0.54%	2.56%	1.66%	0.42%	0.00%
Trans	0.43%	0.32%	0.26%	0.27%	0.35%	0%	0.28%	0.44%
Asian	37.59%	37.65%	36.86%	36.62%	57.6%	57.62%	55.08%	52.85%
Black	2.77%	2.63%	2.79%	2.70%	1.18%	0.84%	1.43%	0.44%
Hawaiian or Pacific Islander	0.00%	0.08%	0.00%	0.04%	0%	0%	0.00%	0.00%
Hispanic or Latino	4.91%	3.25%	3.05%	3.66%	2.47%	1.96%	1.72%	1.97%
Native American or Alaska Native	0.04%	0.16%	0.00%	0.08%	0.12%	0.42%	0.14%	0.00%
Two or More	3.64%	3.88%	3.32%	3.12%	3.06%	3.92%	3.29%	2.63%
Two or More URM	7.28%	5.65%	3.76%	4.63%	2.83%	3.36%	2.43%	2.19%
White, Caucasian, Middle Eastern, North African, Arab	43.77%	46.71%	50.22%	49.15%	32.74%	31.89%	35.91%	39.91%

Question	**Response**
What are three key observations or takeaways you can make from the data table, which includes information regarding students taking introductory computer science courses and students in an upper level computer science course (EECS 376)?	From the data table we can see that there is an overall majority of male- identifying students who take EECS 376 over other genders. We can also see that a majority of Asian students remain in the class. We can also see that by the end of the class, less female students and students of minority groups are still in the class.

Submit worksheet

The team is responsible for submitting to Gradescope by the due date listed on the first page. We strongly recommend submitting your worksheet the same day that you meet as a pod!

The Scribe is responsible for making the final submission to Gradescope. You need to add your other pod members to the submission after it is uploaded using this link: